

Priority Documentation

Preemptive Priority Scheduling:

Correct Process Selection:

we select the ready process with the smallest priority number (smaller = higher priority)

We can preempt the running process if a new process has an even smaller priority number

Process	Arrival	Burst	Priority
P1	0	6	3
P2	2	3	1

t=0: P1 starts. t=2: P2 arrives with priority 1 < running P1's priority 3 => preempt P1, select P2.

t=5: P2 finishes. P1 resumes (remaining 4). t=9: P1 finishes.

Arrival Handling:

- 1-If the new priority is equal or larger =>no preemption, just add to ready queue.
- 2-If CPU idle → jump to next arrival and re-evaluate
- 3-If the new priority is smaller (higher priority) → preempt the running process and start the new one.

Process	Arrival	Burst	Priority
P1	0	5	2
P2	1	2	1

- 1) t=0: CPU idle, P1 arrives => start P1.

Priority Documentation

- 2) $t=1$: P2 arrives. Compare: P2 priority 1 < P1 priority 2 $\Rightarrow$ preemption.
 - 3) P1 stopped (3 remaining), P2 starts immediately.
 - 4) $t=3$: P2 finishes. Ready queue: P1 $\Rightarrow$ resume P1. 4) $t=6$: P1 finishes.
-

Ties:

When multiple ready processes have the same priority, the one that arrived earlier runs first.

If arrival times also equal $\Rightarrow$ smaller PID first.

No preemption occurs between equal priority processes.

Process	Arrival	Burst	Priority
P1	0	4	2
P2	0	3	2

At $t = 0$: both P1 and P2 are ready and have the same priority. first rule:

Earliest arrival — both arrived at 0 $\rightarrow$ still tied. second rule: Smallest

process ID — P1 < P2 $\rightarrow$ P1 runs first

Completion Behavior:

When a process finishes (remaining burst = 0), its TAT, WT, and RT are calculated.

Then the CPU selects the next ready process based on priority (and tie-breaking rules),
not based on those metrics.

Algorithm-Specific Rules:

Priority Documentation

Preemptive Priority Scheduling – Preempt only when a new process has a higher priority (smaller number). No burst-time comparison is needed. Just priority rules

Pseudocode(preemptive)

/*n -> number of processes, arrival [i] -> arrive time of process i, burst [i] -> burst time of process i*/

Algorithm PreemptivePriorityScheduling (n, arrival, burst, priority)

{

 // initialize variables

 remaining $\leftarrow$ Array of size n;

 WT $\leftarrow$ Array of size n (Initialized to 0);

 TAT $\leftarrow$ Array of size n (Initialized to 0);

 RT $\leftarrow$ Array of size n (Initialized to 0);

 start_time $\leftarrow$ Array of size n (Initialized to -1);

 completed_count $\leftarrow$ 0;

 current_time $\leftarrow$ 0;

 For i := 0 to n-1 step 1 do {

 remaining[i] $\leftarrow$ burst[i];

 }

 While completed_count < n do {

 best_process $\leftarrow$ -1;

 For i := 0 to n-1 step 1 do {

 If arrival[i] $\leq$ current_time AND remaining[i] > 0 then {

Priority Documentation

```
If (best_process = -1) OR (priority[i] < priority[best_process]) then {  
    best_process ← i;  
}  
  
/* check if two processes have the same priority → choose earlier  
   arrival time*/  
  
Else If (priority[i] = priority[best_process]) then {  
    If (arrival[i] < arrival[best_process]) then {  
        best_process ← i;  
    }  
  
    /*check if two processes have the same arrival time -> choose the smallest  
       index(ID)*/  
  
    Else If (arrival[i] = arrival[best_process]) AND (i < best_process) then {  
        best_process ← i;  
    }  
}  
  
}  
  
// handle CPU Idle State  
  
If best_process = -1 then {  
    current_time ← current_time + 1;  
}  
  
Else {
```

Priority Documentation

// execution (1 unit of time for Preemption)

If start_time[best_process] = -1 then {

start_time[best_process] $\leftarrow$ current_time;

RT[best_process] $\leftarrow$ start_time[best_process] - arrival[best_process];

}

remaining[best_process] $\leftarrow$ remaining[best_process] - 1;

current_time $\leftarrow$ current_time + 1;

// matrices calculation

If remaining[best_process] = 0 then {

completed_count $\leftarrow$ completed_count + 1;

TAT[best_process] $\leftarrow$ current_time - arrival[best_process];

WT[best_process] $\leftarrow$ TAT[best_process] - burst[best_process];

}

}

}

Write(WT, TAT, RT);

}

Pseudocode(non-preemptive)

Algorithm nonPreemptivePriority (n, arrival, burst, priority) {

// initialize data

WT $\leftarrow$ Array of size n (Initialized to 0);

Priority Documentation

TAT $\leftarrow$ Array of size n (Initialized to 0);

RT $\leftarrow$ Array of size n (Initialized to 0);

isFinished $\leftarrow$ Array of size n (Initialized to 0);

complete_count $\leftarrow$ 0;

current_time $\leftarrow$ 0;

While completed_count < n do {

 best_process $\leftarrow$ -1;

 // find the best process that has arrived and not finished

 For i := 0 to n-1 step 1 do {

 If arrival[i] $\leq$ current_time AND is_finished[i] = false then {

 If best_process = -1 OR priority[i] < priority[best_process] then {

 best_process $\leftarrow$ i;

 }

 Else If priority[i] = priority[best_process] then {

 If arrival[i] < arrival[best_process] then {

 best_process $\leftarrow$ i;

 }

 Else If arrival[i] = arrival[best_process] AND I < best_process then {

 best_process $\leftarrow$ i;

 }

 }

Priority Documentation

```
    }  
}  
  
// If no process is ready, increase current time by 1  
  
If best_process = -1 then {  
    current_time  $\leftarrow$  current_time + 1;  
}  
  
Else {  
    RT[best_process]  $\leftarrow$  current_time - arrival[best_process];  
    current_time += burst[best_process];  
    TAT[best_process] = current_time - arrival[best_process];  
    WT[best_process] = TAT[best_process] - burst[best_process];  
    isFinished[best_process] = true;  
    completed_count++;  
}  
}  
  
Write(WT, TAT, RT);  
}
```